SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07
NAME: K MAHESH
REG.NO:192312376

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1.

Problem Understanding

Inputs

- Total Data Received = 8000 bits
- Time = 4 seconds
- Packets Sent = 200
- Packets Received = 180

Outputs

- Throughput (bits per second)
- Packet Loss (%)
- Network Classification
- Performance Summary

Constraints

- Time must not be zero.
- Packets sent must not be zero.

Formula

Throughput=DataReceivedTimeThroughput=\frac{DataReceived}{Time}

T h r o u g h p u t = T i m e D a t a R e c e i v e d

PacketLoss=PacketsSent−PacketsReceivedPacketsSent×100PacketLoss=\frac{Pack e t s S e n t - P a c k e t s R e c e i v e d } { P a c k e t s S e n t }

\times100PacketLoss=PacketsSentPacketsSent−PacketsReceived×100

```
BEGIN

    SET DataReceived = 8000
    SET Time = 4
    SET PacketsSent = 200
    SET PacketsReceived = 180

    IF Time = 0 THEN
        PRINT "Error: Time cannot be zero."
        STOP
    ENDIF

    IF PacketsSent = 0 THEN
        PRINT "Error: Packets sent cannot be zero."
        STOP
    ENDIF

    Throughput ← DataReceived / Time
    PacketLoss ← ((PacketsSent - PacketsReceived) / PacketsSent) × 100

    IF Throughput > 1500 AND PacketLoss < 10 THEN
        Status ← "Efficient"
    ELSE
        Status ← "Inefficient"
    ENDIF

    PRINT "----- Network Performance Summary -----"
    PRINT "Data Received:", DataReceived, "bits"
    PRINT "Time:", Time, "seconds"
    PRINT "Packets Sent:", PacketsSent
    PRINT "Packets Received:", PacketsReceived
    PRINT "Throughput:", Throughput, "bps"
    PRINT "Packet Loss:", PacketLoss, "%"
    PRINT "Network Status:", Status

END
```

Sample Output

Metric	Value
Throughput	2000 bps
Packet Loss	10%
Classificatio n	Inefficie nt

2.

Solution

Problem Understanding

Inputs

- Number of network links
- Bandwidth utilization of each link

Outputs

- Current utilization
- Overloaded link alerts
- Backup link recommendation

Constraint

- Monitor continuously.
- Threshold = 80%

```
BEGIN

  INPUT NumberOfLink
  DECLARE Utilization[NumberOfLinks]
  WHILE TRUE
    PRINT "Checking Network Links..."
    FOR i ← 1 TO NumberOfLinks
      INPUT Utilization[i]
      PRINT "Link", i, "Utilization:", Utilization[i], "%"
      IF Utilization[i] > 80 THEN
        PRINT "Warning: Link", i, "is overloaded."
        PRINT "Recommendation: Switch traffic to Backup Link."
      ELSE
        PRINT "Link", i, "is operating normally."
      ENDIF
    ENDFOR
    WAIT 10 Second
  ENDWHILE
END
```

How It Supports Efficient Bandwidth Management

- Continuously monitors all network links.
- Stores utilization values in an array.
- Detects overload when utilization exceeds 80%.
- Suggests switching traffic to backup links.
- Prevents congestion by distributing traffic.
- Improves network performance and reliability.

This satisfies the requirement of using loops, arrays, conditions, and continuous monitoring.

3.

Solution

Problem Understanding

A company has six branches connected through multiple network links. The administrator must select the best communication path based on:

- Bandwidth
- Delay
- Utilization

The algorithm must:

- Monitor continuously.
- Calculate link quality.
- Detect overloaded or failed links.
- Automatically choose an alternate path.
- Generate alerts.

Link Quality Formula

A simple scoring formula:

$QualityScore=Bandwidth-Delay-Utilization$QualityScore=Bandwidth-Delay-UtilizationQualityScore=Bandwidth−Delay−Utilization

Higher scores indicate better paths.

```
BEGIN
    INPUT NumberOfLinks
    DECLARE Bandwidth[NumberOfLinks]
    DECLARE Delay[NumberOfLinks]
    DECLARE Utilization[NumberOfLinks]
    DECLARE QualityScore[NumberOfLinks]
    WHILE TRUE
        PRINT "Network Monitoring Started"
        FOR i ← 1 TO NumberOfLinks
```

```
        INPUT Bandwidth[i]
        INPUT Delay[i]
        INPUT Utilization[i]
        QalityScore[i] ← Bandwidth[i] - Delay[i] - Utilization[i]
    ENDFOR
     BestLink ← 1
     FOR i ← 2 TO NumberOfLinks
        IF QualityScore[i] > QualityScore[BestLink] THEN
           BestLink ← i
        ENDIF
     ENDFOR
    PRINT "Best Communication Link:", BestLink
     FOR i ← 1 TO NumberOfLinks
        IF Utilization[i] > 80 THEN
           PRINT "Warning: Link", i, "is overloaded."
           PRINT "Traffic shifted to alternate path."
        ENDIF
        IF Bandwidth[i] = 0 THEN
           PRINT "Alert: Link", i, "has failed."
           PRINT "Automatic rerouting activated."
        ENDIF
     ENDFOR
    WAIT 10 Seconds
  ENDWHILE
END
```

Example

Link	**Bandwidth**	**Delay**	**Utilization**	**Quality Score**
1	100	20	30	50
2	120	15	20	85
3	90	25	40	25

How the Algorithm Improves the Network

1. Network Reliability

- Detects failures immediately.
- Automatically reroutes traffic.

2. Routing Efficiency

- Chooses the highest-quality path.
- Considers bandwidth, delay, and utilization together.

3. Fault Tolerance

- Uses alternate paths during failures.
- Reduces communication interruptions.

4. Congestion Prevention

- Detects links exceeding 80% utilization.
- Prevents overload by shifting traffic.

5. Adaptability

- Continuously updates decisions as network conditions change

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/ modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient